Reviewer Pack — Matroid Rank Separation in Read-Twice Branching Programs

Entry 99773cf419c5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 13:41:26 UTC

Matroid Rank Separation in Read-Twice Branching Programs

Entry ID: `99773cf419c5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 13:41:26 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For any read-twice branching program P over n variables, the rank of the matroid M_P derived from its transition matrix satisfies rank(M_P) ≥ Ω(n), whereas for read-once programs, rank(M_P) = O(log n).

Rationale (proposer's reasoning):

> Matroid rank captures the combinatorial structure of dependencies in branching programs. Read-twice programs introduce persistent dependencies across paths, inflating matroid rank compared to read-once programs, which have acyclic dependency graphs.

Taxonomy category: `BP_READTWICE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9fafe6c00c2061b0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot row
        max_row = i
        for r in range(i + 1, rows):
            if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                max_row = r
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below pivot
        pivot = matrix[i][i]
        if pivot == 0:
            continue
        for j in range(i, cols):
            matrix[i][j] /= pivot

        for r in range(rows):
            if r != i and matrix[r][i] != 0:
                factor = matrix[r][i]
                for j in range(i, cols):
                    matrix[r][j] -= factor * matrix[i][j]
    rank = sum(1 for row in matrix if any(val != 0 for val in row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)

    def generate_bp(n):
        bp = []
        for i in range(n):
            bp.append(random.choice(['R', 'O']))
        return bp

    def is_read_once(bp):
        return all(x == 'O' for x in bp)

    def transition_matrix(bp):
        m = [[0] * n for _ in range(n)]
        for i in range(n):
```

```
            if bp[i] == 'R':
                j = random.randint(0, i - 1) if i > 0 else 0
                m[i][j] = 1
            elif bp[i] == 'O':
                j = random.randint(i + 1, n - 1)
                m[j][i] = 1
        return m

    def matroid_rank(matrix):
        return gaussian_elimination(matrix)

    read_once_count = 0
    total_rank = 0

    for _ in range(30):
        bp = generate_bp(n)
        if is_read_once(bp):
            read_once_count += 1
            rank = matroid_rank(transition_matrix(bp))
            if rank > 0.1 * n:
                continue
            else:
                return {
                    "metric_name": "matroid_rank",
                    "metric_value": rank,
                    "instances_tested": 30,
                    "conjecture_holds": False,
                    "counterexample": f"Read-once BP with rank {rank} <= 0.1n"
                }
        else:
            rank = matroid_rank(transition_matrix(bp))
            if rank < 0.8 * n:
                continue
            total_rank += rank

    avg_rank = total_rank / (30 - read_once_count)
    return {
        "metric_name": "matroid_rank",
        "metric_value": avg_rank,
        "instances_tested": 30,
        "conjecture_holds": True if avg_rank >= 0.8 * n else False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    avg_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={avg_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_rank} std=0.0 support_fraction={support_fraction}")
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = r["counterexample"]
                first_failing_seed = seed
                break

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3341c286.py", line 109, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3341c286.py", line 89, in run_trial
    rank = matroid_rank(transition_matrix(bp))
                        ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3341c286.py", line 63, in transition_ma
    j = random.randint(i + 1, n - 1)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 319, in randrange
    raise ValueError(f"empty range in randrange({start}, {stop})")
ValueError: empty range in randrange(33, 33)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to invalid randint range, preventing data collection. The error suggests a code bug rather than a conjecture failure. | next: Fix the test code to handle edge cases where i+1 ≥ n-1, then re-run with proper parameter ranges.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112865
2	propose	ollama_remote	qwen3:8b	0	0	38092
3	propose	ollama_remote	qwen3:8b	0	0	115091
4	propose	ollama_remote	qwen3:8b	0	0	113245
5	preregistration	ollama_remote	qwen3:8b	0	0	27399
6	novelty	ollama_remote	qwen3:8b	0	0	24801
7	novelty	ollama_remote	qwen3:8b	0	0	18826
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13509
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11033
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9843
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11135
12	judge	ollama_remote	qwen3:8b	0	0	25090

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 520929 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/99773cf419c5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/99773cf419c5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/99773cf419c5.tar.gz` (if generated)